

Value-Based RL

CS 185/285

Instructor: Sergey Levine
UC Berkeley



Part 1:

Actor-critic without the actor

Review: actor-critic with a replay buffer

off-policy actor-critic algorithm:

1. get $(\mathbf{s}_i, \mathbf{a}_i, \mathbf{s}'_i)$ by taking one step with $\mathbf{a} \sim \pi_\theta(\mathbf{a}|\mathbf{s})$
 2. evaluate $y_i = r(\mathbf{s}_i, \mathbf{a}_i) + \gamma \mathbb{E}_{\mathbf{a}' \sim \pi_\theta(\mathbf{a}'|\mathbf{s}'_i)} [\hat{Q}_\phi^\pi(\mathbf{s}'_i, \mathbf{a}')]$
 3. update ϕ using $\nabla_\phi \sum_{i=1}^B \|\hat{Q}_\phi^\pi(\mathbf{s}_i, \mathbf{a}_i) - y_i\|^2$
 - ~~4. $\nabla_\theta J(\theta) \approx \sum_i \mathbb{E}_{\mathbf{a} \sim \pi_\theta(\mathbf{a}|\mathbf{s}_i)} [\nabla_\theta \log \pi_\theta(\mathbf{a}|\mathbf{s}_i) \hat{Q}_\phi^\pi(\mathbf{s}_i, \mathbf{a})]$~~
 - ~~5. $\theta \leftarrow \theta + \alpha \nabla_\theta J(\theta)$~~
- Diagram illustrating the off-policy actor-critic algorithm with a replay buffer:
- The first three steps (1, 2, 3) are grouped by a green bracket and labeled "store $\mathbf{s}_i, \mathbf{a}_i, \mathbf{s}'_i$ ".
 - The last two steps (4, 5) are crossed out with a red line and labeled "use batch $\{(\mathbf{s}_i, \mathbf{a}_i, \mathbf{s}'_i)\}_{i=1}^B$ from buffer".
 - A gray cylinder labeled "replay buffer" is shown, receiving data from step 1 and providing data to step 4.

$$y_i = r(\mathbf{s}_i, \mathbf{a}_i) + \gamma \mathbb{E}_{\mathbf{a}' \sim \pi_\theta(\mathbf{a}'|\mathbf{s}'_i)} [\hat{Q}_\phi^\pi(\mathbf{s}'_i, \mathbf{a}')]$$



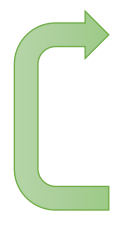
$$r(\mathbf{s}_i, \mathbf{a}_i) + \gamma \max_{\mathbf{a}'} \hat{Q}_\phi^\pi(\mathbf{s}'_i, \mathbf{a}')$$

a funny idea:

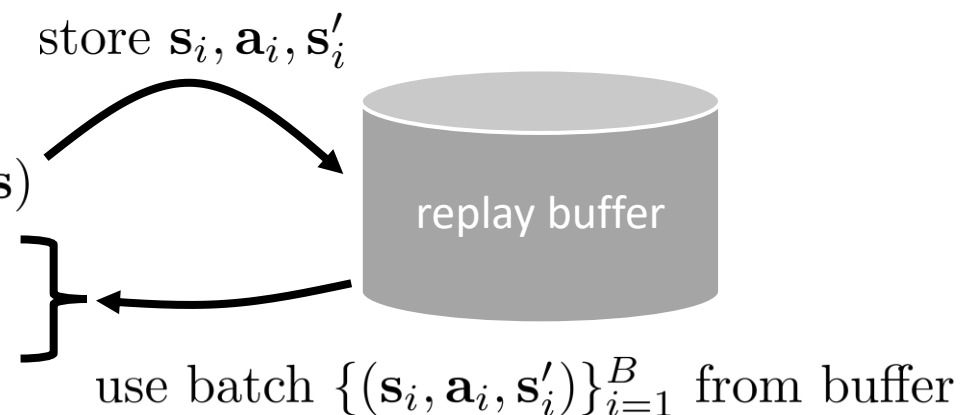
$$\text{set } \pi_\theta(\mathbf{a}|\mathbf{s}) = \begin{cases} 1 & \text{if } \mathbf{a} = \arg \max \hat{Q}_\phi^\pi(\mathbf{s}, \mathbf{a}) \\ 0 & \text{otherwise} \end{cases}$$

The resulting algorithm

off-policy “critic” algorithm (Q-learning):

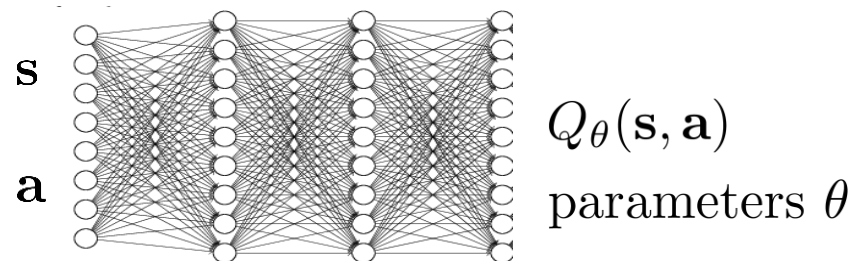
- 
1. get $(\mathbf{s}_i, \mathbf{a}_i, \mathbf{s}'_i)$ by taking one step with $\mathbf{a} \sim \pi_\theta(\mathbf{a}|\mathbf{s})$
 2. evaluate $y_i = r(\mathbf{s}_i, \mathbf{a}_i) + \gamma \max_{\mathbf{a}'} Q_\theta(\mathbf{s}'_i, \mathbf{a}')$
 3. update θ using $\nabla_\theta \sum_{i=1}^B \|Q_\theta(\mathbf{s}_i, \mathbf{a}_i) - y_i\|^2$

$$\pi_\theta(\mathbf{a}|\mathbf{s}) = \begin{cases} 1 & \text{if } \mathbf{a} = \arg \max Q_\theta(\mathbf{s}, \mathbf{a}) \\ 0 & \text{otherwise} \end{cases}$$



We could just stop here

But it helps to understand more...



Part 2:

Policy iteration and dynamic programming

解决确定性环境的最优决策问题
提供小规模离散场景的高效求解方案

Policy iteration

策略迭代是通过「评估当前策略→优化策略」的循环，逐步逼近最优策略

原则：基于评估得到的Q值，选择每个状态下能最大化Q值的动作，生成新策略 π'

High level idea:

policy iteration algorithm:

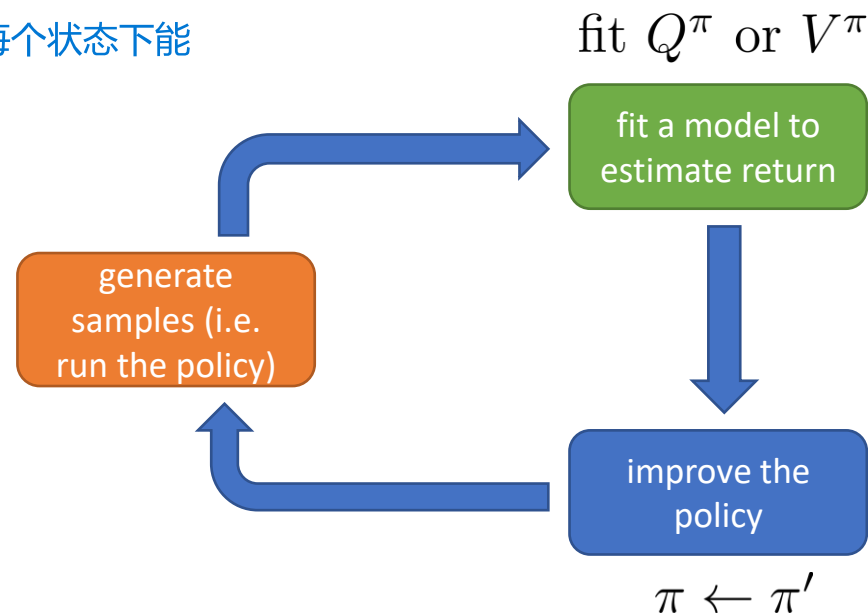
- ➡ 1. evaluate $Q^\pi(\mathbf{s}, \mathbf{a})$ ← how to do this?
- ➡ 2. set $\pi \leftarrow \pi'$ 策略更新：将当前策略替换为新策略，即 $\pi \leftarrow \pi'$ ，然后回到策略评估步骤，继续迭代

$$\pi'(\mathbf{a}_t | \mathbf{s}_t) = \begin{cases} 1 & \text{if } \mathbf{a}_t = \arg \max_{\mathbf{a}_t} Q^\pi(\mathbf{s}_t, \mathbf{a}_t) \\ 0 & \text{otherwise} \end{cases}$$

as before: $Q^\pi(\mathbf{s}, \mathbf{a}) = r(\mathbf{s}, \mathbf{a}) + \gamma E[V^\pi(\mathbf{s}')]$

let's evaluate $V^\pi(\mathbf{s})!$

不再用梯度来更新策略，而是用最大化价值的action决定的下一步策略来更新策略



Dynamic programming

Let's assume we know $p(\mathbf{s}'|\mathbf{s}, \mathbf{a})$, and $\mathbf{s}$ and $\mathbf{a}$ are both discrete (and small)

0.2	0.3	0.4	0.3
0.3	0.3	0.5	0.3
0.4	0.4	0.6	0.4
0.5	0.5	0.7	0.5

16 states, 4 actions per state

can store full $V^\pi(\mathbf{s})$ in a table!

$\mathcal{T}$ is $16 \times 16 \times 4$ tensor transition tensor

bootstrapped update: $V^\pi(\mathbf{s}) \leftarrow E_{\mathbf{a} \sim \pi(\mathbf{a}|\mathbf{s})}[r(\mathbf{s}, \mathbf{a}) + \gamma E_{\mathbf{s}' \sim p(\mathbf{s}'|\mathbf{s}, \mathbf{a})}[V^\pi(\mathbf{s}')]]$



just use the current estimate here

$$\pi'(\mathbf{a}_t|\mathbf{s}_t) = \begin{cases} 1 & \text{if } \mathbf{a}_t = \arg \max_{\mathbf{a}_t} Q^\pi(\mathbf{s}_t, \mathbf{a}_t) \\ 0 & \text{otherwise} \end{cases} \longrightarrow \text{deterministic policy } \pi(\mathbf{s}) = \mathbf{a}$$

更新策略方式：在状态 $\mathbf{s}_t$ 下，只保留能最大化动作价值的动作，其他动作的选择概率设为0，生成新的确定性策略

simplified: $V^\pi(\mathbf{s}) \leftarrow r(\mathbf{s}, \pi(\mathbf{s})) + \gamma E_{\mathbf{s}' \sim p(\mathbf{s}'|\mathbf{s}, \pi(\mathbf{s}))}[V^\pi(\mathbf{s}')]$

Policy iteration with dynamic programming

policy iteration:

- 
1. evaluate $V^\pi(\mathbf{s})$
 2. set $\pi \leftarrow \pi'$

$$\pi'(\mathbf{a}_t|\mathbf{s}_t) = \begin{cases} 1 & \text{if } \mathbf{a}_t = \arg \max_{\mathbf{a}_t} Q^\pi(\mathbf{s}_t, \mathbf{a}_t) \\ 0 & \text{otherwise} \end{cases}$$

policy evaluation:

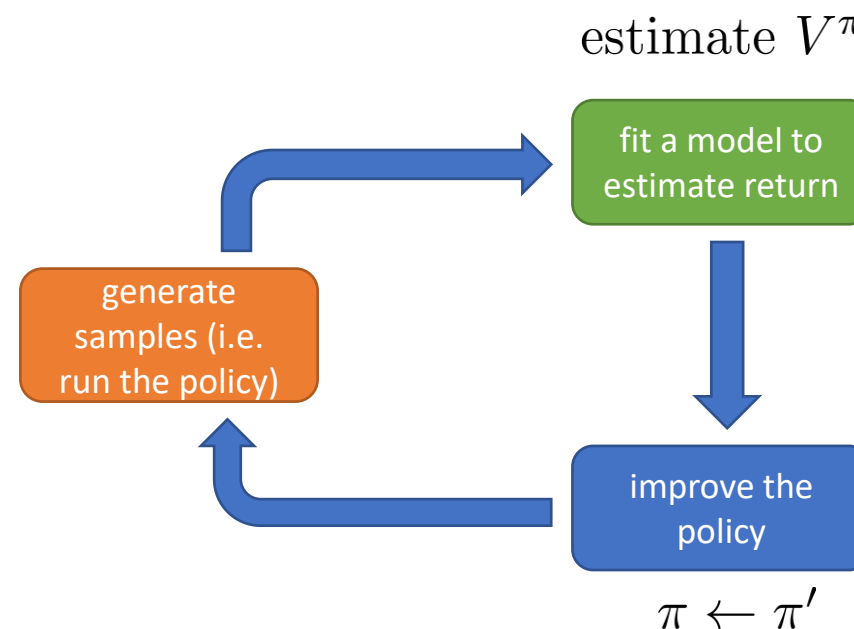

$$V^\pi(\mathbf{s}) \leftarrow r(\mathbf{s}, \pi(\mathbf{s})) + \gamma E_{\mathbf{s}' \sim p(\mathbf{s}'|\mathbf{s}, \pi(\mathbf{s}))} [V^\pi(\mathbf{s}')]]$$

0.2	0.3	0.4	0.3
0.3	0.3	0.5	0.3
0.4	0.4	0.6	0.4
0.5	0.5	0.7	0.5

16 states, 4 actions per state

can store full $V^\pi(\mathbf{s})$ in a table!

$\mathcal{T}$ is $16 \times 16 \times 4$ tensor



该方法依赖动态规划的特性，仅适用于以下场景：
状态 $\mathbf{s}$ 和动作 $\mathbf{a}$ 均为离散且规模较小
已知精确的状态转移概率

Even simpler dynamic programming

$$\pi'(\mathbf{a}_t|\mathbf{s}_t) = \begin{cases} 1 & \text{if } \mathbf{a}_t = \arg \max_{\mathbf{a}_t} Q^\pi(\mathbf{s}_t, \mathbf{a}_t) \\ 0 & \text{otherwise} \end{cases}$$

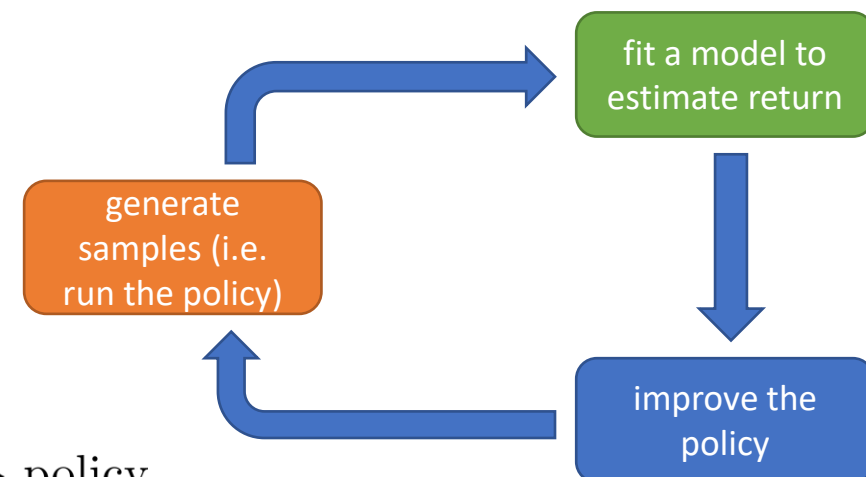
let's evaluate $Q^\pi(\mathbf{s})$!

	a			
s	$Q(\mathbf{s}, \mathbf{a})$		$Q(\mathbf{s}, \mathbf{a})$	$Q(\mathbf{s}, \mathbf{a})$
	$Q(\mathbf{s}, \mathbf{a})$	$Q(\mathbf{s}, \mathbf{a})$		$Q(\mathbf{s}, \mathbf{a})$
		$Q(\mathbf{s}, \mathbf{a})$	$Q(\mathbf{s}, \mathbf{a})$	$Q(\mathbf{s}, \mathbf{a})$
	$Q(\mathbf{s}, \mathbf{a})$	$Q(\mathbf{s}, \mathbf{a})$	$Q(\mathbf{s}, \mathbf{a})$	
	$Q(\mathbf{s}, \mathbf{a})$		$Q(\mathbf{s}, \mathbf{a})$	$Q(\mathbf{s}, \mathbf{a})$
	$Q(\mathbf{s}, \mathbf{a})$	$Q(\mathbf{s}, \mathbf{a})$		$Q(\mathbf{s}, \mathbf{a})$

$\arg \max_{\mathbf{a}} Q(\mathbf{s}, \mathbf{a}) \rightarrow \text{policy}$

approximates the new value!

$$Q^\pi(\mathbf{s}, \mathbf{a}) \leftarrow r(\mathbf{s}, \mathbf{a}) + \gamma E_{\mathbf{s}' \sim p(\mathbf{s}'|\mathbf{s}, \mathbf{a})} [V^\pi(\mathbf{s}')]]$$



skip the policy and compute values directly!

value iteration algorithm:

1. set $Q(\mathbf{s}, \mathbf{a}) \leftarrow r(\mathbf{s}, \mathbf{a}) + \gamma E[V(\mathbf{s}')]]$
2. set $V(\mathbf{s}) \leftarrow \max_{\mathbf{a}} Q(\mathbf{s}, \mathbf{a})$

传统策略迭代需要「评估策略→优化策略」的循环，而该方法直接聚焦价值函数计算，通过迭代更新动作价值和状态价值，最终从 Q^π 中直接提取最优策略，无需显式维护策略 π 。

Part 3:

Fitted value iteration

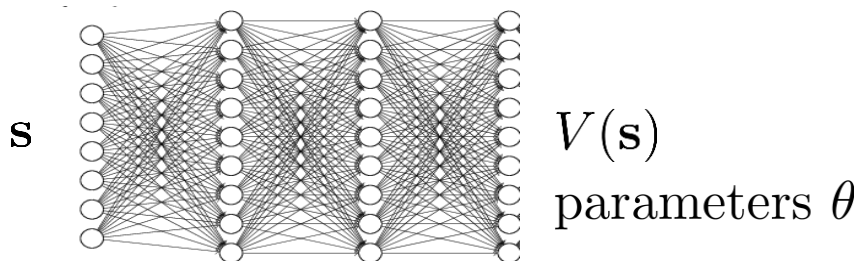
Fitted value iteration

针对的是高维状态空间的强化学习问题

how do we represent $V(\mathbf{s})$?

big table, one entry for each discrete $\mathbf{s}$

neural net function $V : \mathcal{S} \rightarrow \mathbb{R}$



$$\mathcal{L}(\theta) = \frac{1}{2} \left\| V_{\theta}(\mathbf{s}) - \max_{\mathbf{a}} Q^{\pi}(\mathbf{s}, \mathbf{a}) \right\|^2$$

$$\mathbf{s} = 0 : V(\mathbf{s}) = 0.2$$

$$\mathbf{s} = 1 : V(\mathbf{s}) = 0.3$$

$$\mathbf{s} = 2 : V(\mathbf{s}) = 0.5$$

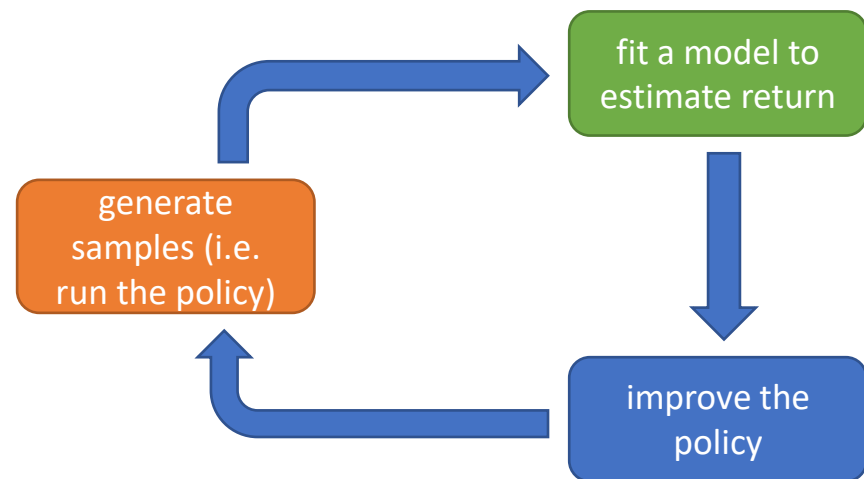


$$|\mathcal{S}| = (255^3)^{200 \times 200}$$

(more than atoms in the universe)

curse of
dimensionality

$$Q^{\pi}(\mathbf{s}, \mathbf{a}) \leftarrow r(\mathbf{s}, \mathbf{a}) + \gamma E_{\mathbf{s}' \sim p(\mathbf{s}'|\mathbf{s}, \mathbf{a})} [V^{\pi}(\mathbf{s}')]]$$



$$V^{\pi}(\mathbf{s}) \leftarrow \max_{\mathbf{a}} Q^{\pi}(\mathbf{s}, \mathbf{a})$$

fitted value iteration algorithm:

1. set $y_i \leftarrow \max_{\mathbf{a}_i} (r(\mathbf{s}_i, \mathbf{a}_i) + \gamma E[V_{\theta}(\mathbf{s}'_i)])$
2. set $\theta \leftarrow \arg \min_{\theta} \frac{1}{2} \sum_i \|V_{\theta}(\mathbf{s}_i) - y_i\|^2$

针对一个极高维的连续状态空间，采用动态规划方案和蒙特卡洛方法迭代的表格存储值函数是不现实的。所以用神经网络对 $\mathbf{s} \rightarrow$ 值函数 V 的映射进行拟合

What if we don't know the transition dynamics?

fitted value iteration algorithm:

该算法的核心是通过样本拟合值函数，无需显式建模状态转移概率
该方法仍需知道不同动作的执行结果，可通过历史样本或交互数据近似期望项。

- 
1. set $y_i \leftarrow \max_{\mathbf{a}_i} (r(\mathbf{s}_i, \mathbf{a}_i) + \gamma E[V_\theta(\mathbf{s}'_i)])$
 2. set $\theta \leftarrow \arg \min_\theta \frac{1}{2} \sum_i \|V_\theta(\mathbf{s}_i) - y_i\|^2$

need to know outcomes
for different actions!

依赖于：1. 要么已知状态转移概率分布 2. 要么通过sampling拟合，但是有偏

Back to policy iteration...

policy iteration:

- 
1. evaluate $Q^\pi(\mathbf{s}, \mathbf{a})$
 2. set $\pi \leftarrow \pi'$

$$\pi'(\mathbf{a}_t | \mathbf{s}_t) = \begin{cases} 1 & \text{if } \mathbf{a}_t = \arg \max_{\mathbf{a}_t} Q^\pi(\mathbf{s}_t, \mathbf{a}_t) \\ 0 & \text{otherwise} \end{cases}$$

policy evaluation:



$$V^\pi(\mathbf{s}) \leftarrow r(\mathbf{s}, \pi(\mathbf{s})) + \gamma E_{\mathbf{s}' \sim p(\mathbf{s}' | \mathbf{s}, \pi(\mathbf{s}))} [V^\pi(\mathbf{s}')]]$$

$$Q^\pi(\mathbf{s}, \mathbf{a}) \leftarrow r(\mathbf{s}, \mathbf{a}) + \gamma E_{\mathbf{s}' \sim p(\mathbf{s}' | \mathbf{s}, \mathbf{a})} [Q^\pi(\mathbf{s}', \pi(\mathbf{s}'))]$$

can fit this using samples



Can we do the “max” trick again?

拟合Q迭代 (Fitted Q Iteration) 算法:

相比策略迭代: 策略迭代需要显式维护策略, 且依赖完整的状态转移模型来完成策略评估和更新; 拟合Q迭代无需显式策略, 也不需要已知状态转移概率, 仅通过样本数据就能完成学习。

相比拟合值迭代: 拟合值迭代需要通过“max”操作隐式优化策略, 且在计算未来回报期望时, 通常需要依赖状态转移模型或大量采样; 拟合Q迭代通过用下一个状态的最大Q值近似未来回报期望, 完全不需要模拟动作执行, 也不依赖状态转移模型。

policy iteration:

- 
1. evaluate $V^\pi(\mathbf{s})$
 2. set $\pi \leftarrow \pi'$



fitted value iteration algorithm:

- 
1. set $y_i \leftarrow \max_{\mathbf{a}_i} (r(\mathbf{s}_i, \mathbf{a}_i) + \gamma E[V_\theta(\mathbf{s}'_i)])$
 2. set $\theta \leftarrow \arg \min_\theta \frac{1}{2} \sum_i \|V_\theta(\mathbf{s}_i) - y_i\|^2$

no policy, compute value directly

can we do this with Q-values **also**, without knowing the transitions?

这一点和actor-critic算法很不一样

fitted Q iteration algorithm:

doesn't require simulation of actions!

- 
1. set $y_i \leftarrow r(\mathbf{s}_i, \mathbf{a}_i) + \gamma E[V_\theta(\mathbf{s}'_i)]$ ← approximate $E[V(\mathbf{s}'_i)] \approx \max_{\mathbf{a}'} Q_\theta(\mathbf{s}'_i, \mathbf{a}'_i)$
 2. set $\theta \leftarrow \arg \min_\theta \frac{1}{2} \sum_i \|Q_\theta(\mathbf{s}_i, \mathbf{a}_i) - y_i\|^2$

+ works even for off-policy samples 支持离策略样本 (off-policy samples) : 可以利用非当前策略产生的历史数据进行学习, 数据利用率更高

+ only one network, no high-variance policy gradient 结构简单: 仅需一个网络, 避免了策略梯度方法的高方差问题

- no convergence guarantees for non-linear function approximation (more on this later)

非线性函数近似的收敛性无法保证: 当用神经网络等非线性函数近似Q值时, 算法可能无法收敛到最优解 (后续会进一步讲解相关解决方案)

Fitted Q-iteration

full fitted Q-iteration algorithm:

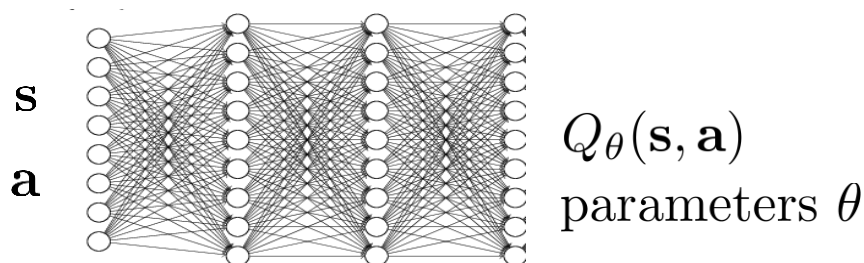
1. collect dataset $\{(\mathbf{s}_i, \mathbf{a}_i, \mathbf{s}'_i)\}$ using some policy
2. set $y_i \leftarrow r(\mathbf{s}_i, \mathbf{a}_i) + \gamma \max_{\mathbf{a}'_i} Q_\theta(\mathbf{s}'_i, \mathbf{a}'_i)$
3. set $\theta \leftarrow \arg \min_\theta \frac{1}{2} \sum_i \|Q_\theta(\mathbf{s}_i, \mathbf{a}_i) - y_i\|^2$
train the new q function, which phi is the weight
- $K \times$

parameters

dataset size N , collection policy

iterations K

gradient steps S



当Q函数收敛到最优Q函数后，策略可以直接通过贪心策略从Q函数导出： $\pi(s)$ 为使得Q function值最大的action，作为最终的策略函数。整个策略函数的更新都是隐式更新

Part 4:

Back to Q-learning

Why is this algorithm off-policy?

full fitted Q-iteration algorithm:

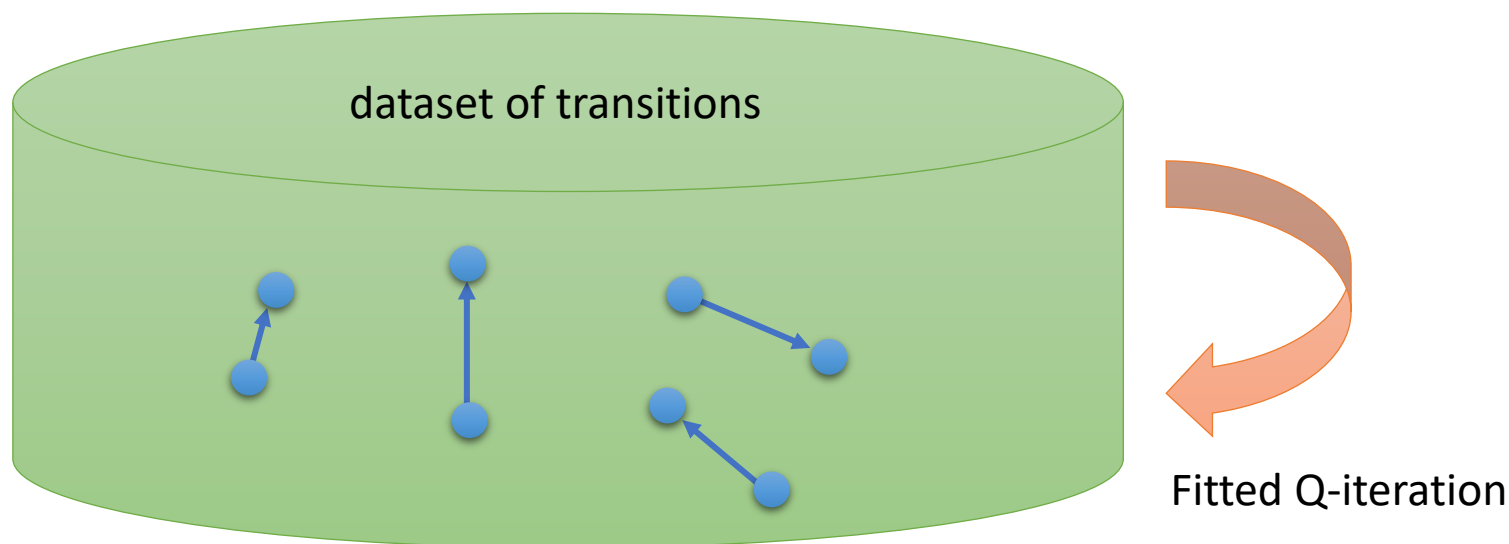
1. collect dataset $\{(\mathbf{s}_i, \mathbf{a}_i, \mathbf{s}'_i)\}$ using some policy
2. set $y_i \leftarrow r(\mathbf{s}_i, \mathbf{a}_i) + \gamma \max_{\mathbf{a}'_i} Q_\theta(\mathbf{s}'_i, \mathbf{a}'_i)$
3. set $\theta \leftarrow \arg \min_\theta \frac{1}{2} \sum_i \|Q_\theta(\mathbf{s}_i, \mathbf{a}_i) - y_i\|^2$

$K \times$

this approximates the value of π' at $\mathbf{s}'_i$

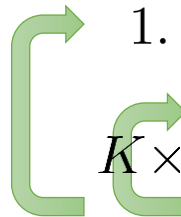
$$\pi'(\mathbf{a}_t | \mathbf{s}_t) = \begin{cases} 1 & \text{if } \mathbf{a}_t = \arg \max_{\mathbf{a}_t} Q^\pi(\mathbf{s}_t, \mathbf{a}_t) \\ 0 & \text{otherwise} \end{cases}$$

given $\mathbf{s}$ and $\mathbf{a}$, transition is independent of π



What is fitted Q-iteration optimizing?

full fitted Q-iteration algorithm:

- 
1. collect dataset $\{(\mathbf{s}_i, \mathbf{a}_i, \mathbf{s}'_i)\}$ using some policy
 2. set $y_i \leftarrow r(\mathbf{s}_i, \mathbf{a}_i) + \gamma \max_{\mathbf{a}'_i} Q_\theta(\mathbf{s}'_i, \mathbf{a}'_i)$ $\longleftarrow$ this max improves the policy (tabular case)
 3. set $\theta \leftarrow \arg \min_\theta \frac{1}{2} \sum_i \|Q_\theta(\mathbf{s}_i, \mathbf{a}_i) - y_i\|^2$
- $\uparrow$
error $\mathcal{E}$

$$\mathcal{E} = \frac{1}{2} E_{(\mathbf{s}, \mathbf{a}) \sim \beta} \left[\left(Q_\theta(\mathbf{s}, \mathbf{a}) - [r(\mathbf{s}, \mathbf{a}) + \gamma \max_{\mathbf{a}'} Q_\theta(\mathbf{s}', \mathbf{a}')] \right)^2 \right]$$

if $\mathcal{E} = 0$, then $Q_\theta(\mathbf{s}, \mathbf{a}) = r(\mathbf{s}, \mathbf{a}) + \gamma \max_{\mathbf{a}'} Q_\theta(\mathbf{s}', \mathbf{a}')$

this is an *optimal* Q-function, corresponding to optimal policy π' :

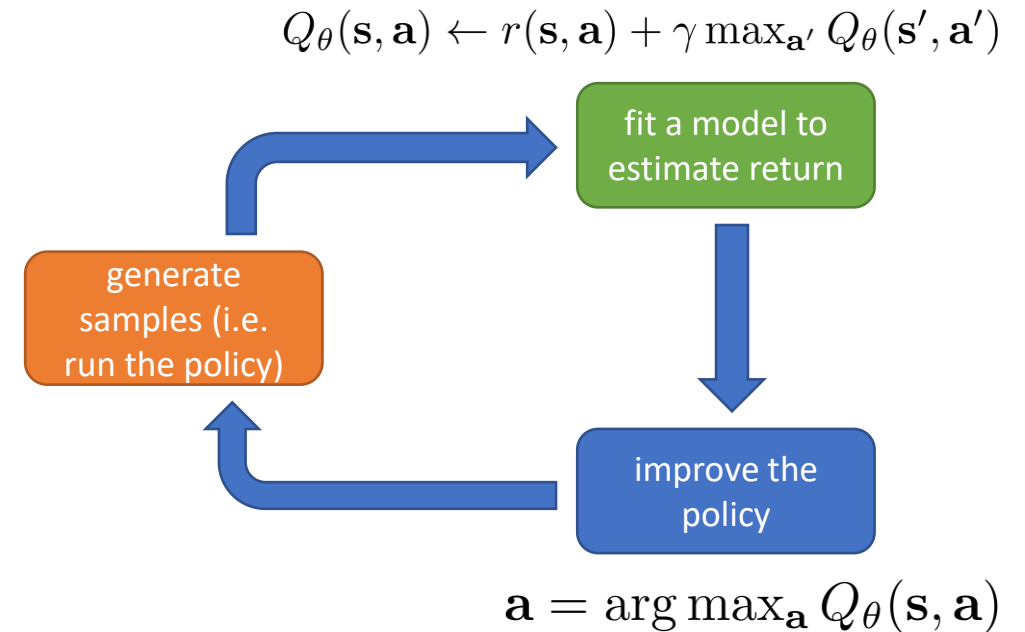
$$\pi'(\mathbf{a}_t | \mathbf{s}_t) = \begin{cases} 1 & \text{if } \mathbf{a}_t = \arg \max_{\mathbf{a}_t} Q_\phi(\mathbf{s}_t, \mathbf{a}_t) \\ 0 & \text{otherwise} \end{cases} \quad \begin{array}{l} \text{maximizes reward} \\ \text{sometimes written } Q^* \text{ and } \pi^* \end{array}$$

most guarantees are lost when we leave the tabular case (e.g., use neural networks)

Online Q-learning algorithms

full fitted Q-iteration algorithm:

1. collect dataset $\{(\mathbf{s}_i, \mathbf{a}_i, \mathbf{s}'_i)\}$ using some policy
2. set $y_i \leftarrow r(\mathbf{s}_i, \mathbf{a}_i) + \gamma \max_{\mathbf{a}'} Q_\theta(\mathbf{s}'_i, \mathbf{a}'_i)$
3. set $\theta \leftarrow \arg \min_{\theta} \frac{1}{2} \sum_i \|Q_\theta(\mathbf{s}_i, \mathbf{a}_i) - y_i\|^2$



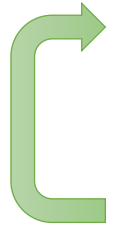
online Q iteration algorithm:

off policy, so many choices here!

1. take some action $\mathbf{a}_i$ and observe $(\mathbf{s}_i, \mathbf{a}_i, \mathbf{s}'_i)$
2. $y_i = r(\mathbf{s}_i, \mathbf{a}_i) + \gamma \max_{\mathbf{a}'} Q_\theta(\mathbf{s}'_i, \mathbf{a}'_i)$
3. $\theta \leftarrow \theta - \alpha \frac{dQ_\theta}{d\theta}(\mathbf{s}_i, \mathbf{a}_i)(Q_\theta(\mathbf{s}_i, \mathbf{a}_i) - y_i)$

Exploration with Q-learning

online Q iteration algorithm:

- 
1. take some action $\mathbf{a}_i$ and observe $(\mathbf{s}_i, \mathbf{a}_i, \mathbf{s}'_i)$
 2. $y_i = r(\mathbf{s}_i, \mathbf{a}_i) + \gamma \max_{\mathbf{a}'} Q_\theta(\mathbf{s}'_i, \mathbf{a}'_i)$
 3. $\theta \leftarrow \theta - \alpha \frac{dQ_\theta}{d\theta}(\mathbf{s}_i, \mathbf{a}_i)(Q_\theta(\mathbf{s}_i, \mathbf{a}_i) - y_i)$

final policy:

$$\pi(\mathbf{a}_t | \mathbf{s}_t) = \begin{cases} 1 & \text{if } \mathbf{a}_t = \arg \max_{\mathbf{a}_t} Q_\theta(\mathbf{s}_t, \mathbf{a}_t) \\ 0 & \text{otherwise} \end{cases}$$

why is this a bad idea for step 1?

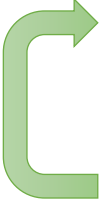
$$\pi(\mathbf{a}_t | \mathbf{s}_t) = \begin{cases} 1 - \epsilon & \text{if } \mathbf{a}_t = \arg \max_{\mathbf{a}_t} Q_\theta(\mathbf{s}_t, \mathbf{a}_t) \\ \epsilon / (|\mathcal{A}| - 1) & \text{otherwise} \end{cases} \quad \text{“epsilon-greedy”}$$

$$\pi(\mathbf{a}_t | \mathbf{s}_t) \propto \exp(Q_\theta(\mathbf{s}_t, \mathbf{a}_t)) \quad \text{“Boltzmann exploration”}$$

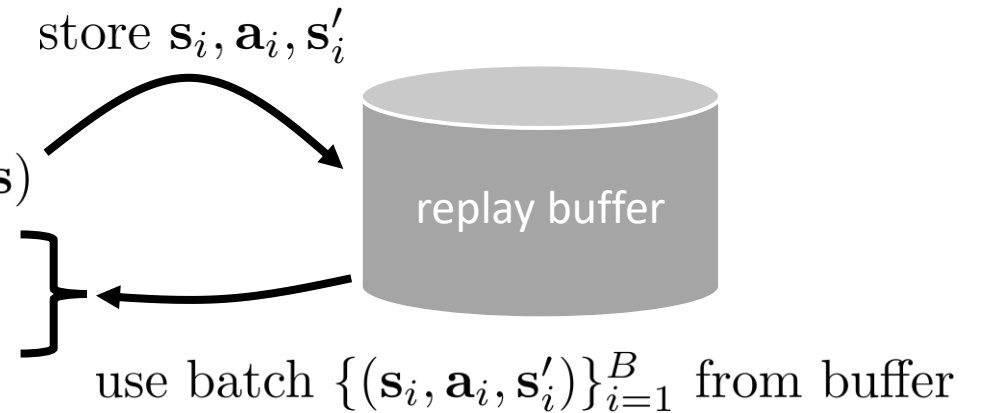
We'll discuss exploration in detail in a later lecture!

Bringing it back together

Q-learning with replay buffer:

- 
1. get $(\mathbf{s}_i, \mathbf{a}_i, \mathbf{s}'_i)$ by taking one step with $\mathbf{a} \sim \pi_\theta(\mathbf{a}|\mathbf{s})$
 2. evaluate $y_i = r(\mathbf{s}_i, \mathbf{a}_i) + \gamma \max_{\mathbf{a}'} Q_\theta(\mathbf{s}'_i, \mathbf{a}')$
 3. update θ using $\nabla_\theta \sum_{i=1}^B \|Q_\theta(\mathbf{s}_i, \mathbf{a}_i) - y_i\|^2$

$$\pi_\theta(\mathbf{a}|\mathbf{s}) = \begin{cases} 1 - \epsilon & \text{if } \mathbf{a} = \arg \max_{\mathbf{a}} Q_\theta(\mathbf{s}, \mathbf{a}) \\ \epsilon / (|\mathcal{A}| - 1) & \text{otherwise} \end{cases}$$



If you try to code this up, it almost certainly won't work

We'll find out how to make it work next time!